

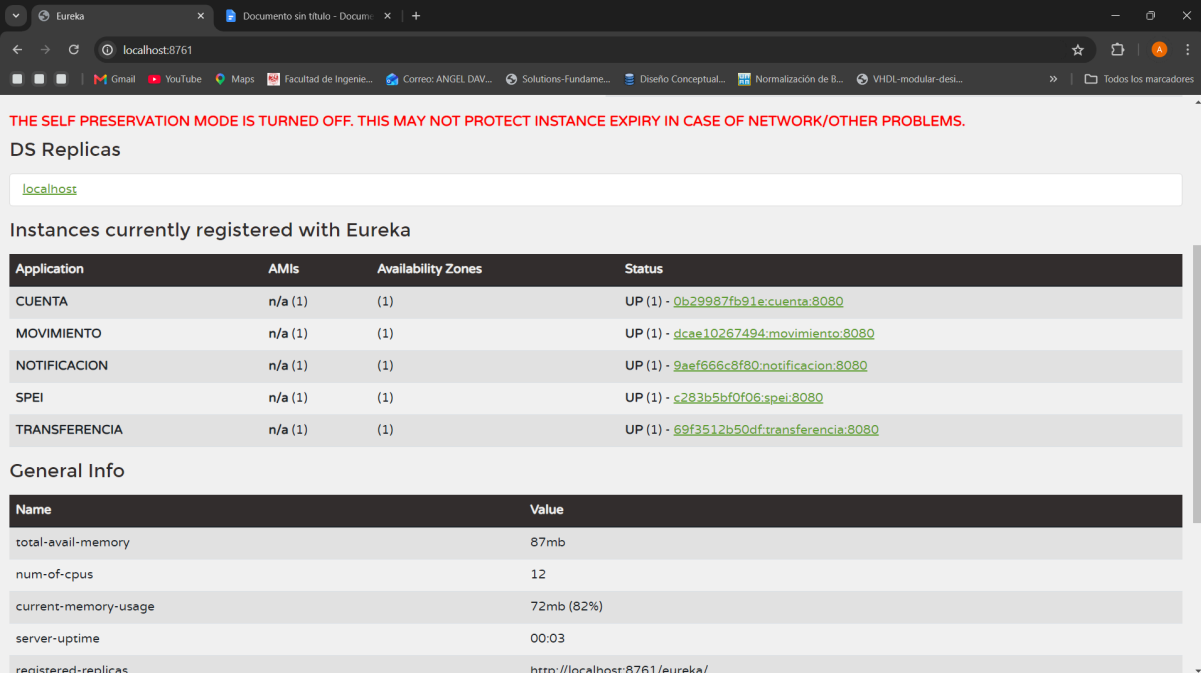
Reporte Laboratorio S07

19:30

Al cambiar llamadas a métodos directos por llamadas HTTP, los servicios dejaron de compartir la misma memoria, por lo que perdimos la garantía de que el otro servicio exista. Ahora dependemos de la red (que puede fallar, tener latencia o desconectarse) y perdimos el "Type Safety" de Java, teniendo que mandar los datos serializados en JSON. Además, ahora necesitamos saber la IP y puerto exacto del otro servicio para hablarle.

20:15

Al inicializar la arquitectura y acceder al servidor de descubrimiento (Eureka) en el puerto 8761, se verificó el correcto registro de los microservicios. En el panel principal, dentro de la sección *Instances currently registered with Eureka*, se confirmó que los servicios fundamentales (CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA) se encuentran operativos, cada uno reportando un estado de **UP (1)**, lo cual indica que están listos para recibir y procesar peticiones dentro de la red.



The screenshot shows the Eureka web interface in a browser. At the top, a red warning message states: "THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS." Below this, the "DS Replicas" section shows "localhost" as the only replica. The "Instances currently registered with Eureka" section contains a table with the following data:

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 0b29987fb91e:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - dcae10267494:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 9aef666c8f80:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - c283b5bf0f06:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 69f3512b50df:transferencia:8080

Below the table, the "General Info" section shows the following details:

Name	Value
total-avail-memory	87mb
num-of-cpus	12
current-memory-usage	72mb (82%)
server-uptime	00:03
registered-replicas	http://localhost:8761/eureka/

El directorio centralizado resuelve el problema de las direcciones estáticas. Si un contenedor colapsa y Docker lo levanta con una IP distinta, o si se instancian nuevas réplicas para soportar más tráfico, el directorio actualiza este mapeo en tiempo real. Esto permite que los microservicios se descubran dinámicamente mediante nombres lógicos, sin necesidad de apagar, reconfigurar o recompilar el código de los clientes que los consumen.

El error HTTP 400 (Bad Request) al apuntar directamente al puerto 8082 ocurrió al intentar consumir el microservicio interno saltándose la puerta de enlace. La petición fue rechazada por el servidor porque carecía de la estructura, los parámetros o los encabezados que el servicio exige, demostrando el riesgo de exponer las APIs de forma directa y la necesidad de usar el Gateway para estandarizar y sanitizar el tráfico entrante.

20:45

Prueba de ruteo del API Gateway (Error 404)

Durante las pruebas de ruteo unificado, se realizó una petición a un recurso inexistente (/api/cuentas/000). La respuesta arrojó un código HTTP 404 (Not Found). Sin embargo, la inclusión de la cabecera X-Instancia en la respuesta confirma que el Gateway enrutó correctamente la solicitud hacia el microservicio destino, demostrando que fue la lógica de negocio de dicho servicio (y no un fallo de red) la que determinó la inexistencia del recurso.

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 0b29987fb91e
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 02:46:17 GMT
```

Prueba de seguridad y bloqueo del API Gateway

Se evaluó la seguridad perimetral del sistema al intentar registrar un movimiento desde el exterior mediante una petición HTTP POST a /api/movimientos. La conexión fue rechazada inmediatamente. Este comportamiento valida que el Gateway actúa como un filtro eficaz, denegando el tráfico hacia rutas internas no expuestas explícitamente y protegiendo así las operaciones privadas del ecosistema.

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
curl: (56) Recv failure: Connection was reset
```

Creación de recursos a través del Gateway

Se validó el flujo de creación de datos ejecutando una petición POST hacia el microservicio de cuentas a través de la puerta de enlace. El sistema respondió con un código HTTP 201 (Created), confirmando la persistencia del dato. Posteriormente, una consulta GET a la misma cuenta devolvió un código HTTP 200 (OK) y el saldo inicial esperado, comprobando la integridad funcional de la API unificada.

```

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d "{\"clabe\":\"$LAB_CLABE\",\"titular\":\"S07\",\"saldoInicial\":0}"

URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
curl -i "$URL"
echo "$URL"
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 0b29987fb91e
Content-Type: application/json
Date: Sat, 10 Oct 2026 02:49:01 GMT

{"id":1,"clabe":"S071791600539","titular":"S07","saldo":0}HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 0b29987fb91e
Content-Type: application/json
Date: Sat, 10 Oct 2026 02:49:03 GMT

{"id":1,"clabe":"S071791600539","titular":"S07","saldo":0.00}http://localhost:80
80/api/cuentas/S071791600539

```

Escalamiento Horizontal del Servicio

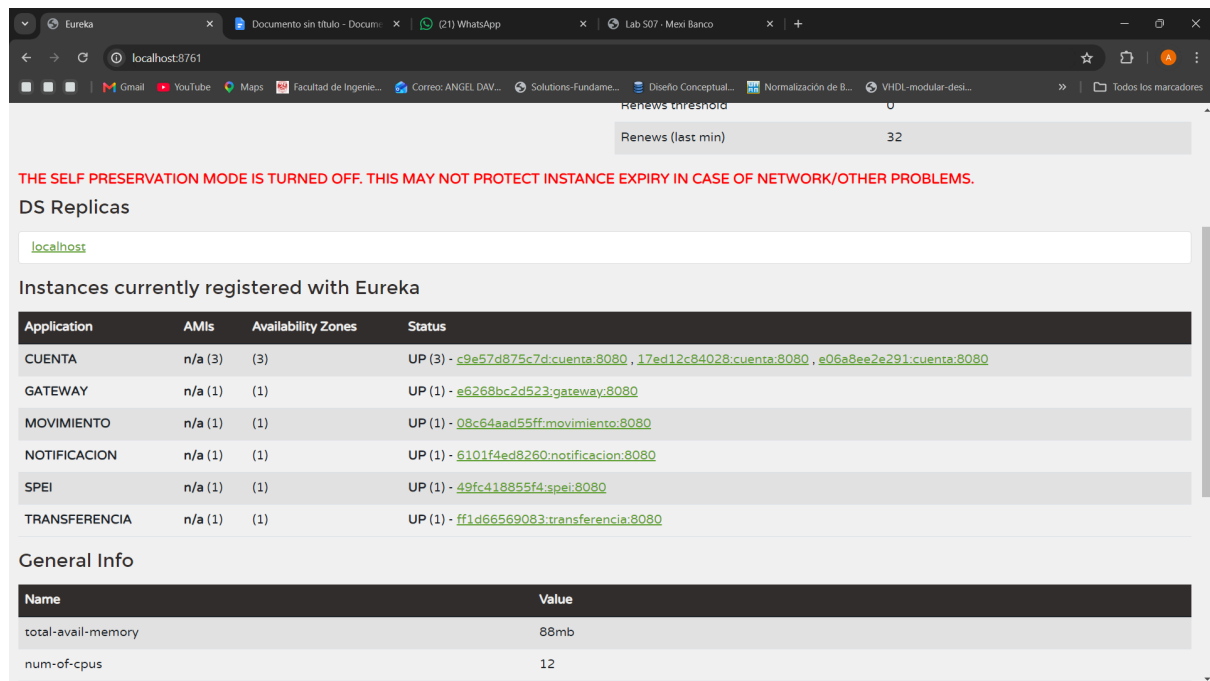
Para dotar al sistema de alta disponibilidad, se modificó la configuración de orquestación para escalar el microservicio cuenta a tres réplicas independientes. Mediante los comandos de monitoreo de Docker se verificó el despliegue exitoso de estas instancias.

```

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
e06a8ee2e291 mexi-banco-v06-cuenta-2 8080/tcp
17ed12c84028 mexi-banco-v06-cuenta-3 8080/tcp
c9e57d875c7d mexi-banco-v06-cuenta-1 8080/tcp

```

Posteriormente, esto se vio reflejado en el servidor de descubrimiento Eureka, el cual actualizó su registro consolidando las instancias con un estado operativo de **UP (3)**



Renews threshold: 0

Renews (last min): 32

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - c9e57d875c7d:cuenta:8080 , 17ed12c84028:cuenta:8080 , e06a8ee2e291:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - e6268bc2d523:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 08c64aad55ff:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 6101f4ed8260:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 49fc418855f4:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - ff1d66569083:transferencia:8080

General Info

Name	Value
total-avail-memory	88mb
num-of-cpus	12

Validación del Balanceador de Carga (Round Robin)

Al realizar múltiples peticiones secuenciales, se analizó la cabecera **X-Instancia** de cada respuesta. Se observó un patrón de alternancia constante entre tres identificadores de contenedor distintos. Esto demuestra que el Gateway, apoyado en el registro de Eureka, implementa con éxito un algoritmo de balanceo de carga tipo *Round Robin*, distribuyendo el tráfico de manera equitativa.

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: c9e57d875c7d
X-Instancia: 17ed12c84028
X-Instancia: e06a8ee2e291
X-Instancia: c9e57d875c7d
X-Instancia: e06a8ee2e291
X-Instancia: 17ed12c84028
X-Instancia: c9e57d875c7d
X-Instancia: e06a8ee2e291
X-Instancia: 17ed12c84028
```

Predicción:

¿Van a fallar todas las consultas, ninguna o algunas? ¿Durante cuánto tiempo?

Se anticipa que al interrumpir un nodo, habría fallas intermitentes afectando solo a las peticiones asignadas a ese nodo específico, perdurando únicamente hasta que el directorio detectara la caída.

21:07

Prueba de Tolerancia a Fallos

Al forzar la detención abrupta de una de las réplicas, se indujo un estado degradado temporal donde las peticiones exitosas (HTTP 200) se intercalaron con errores (HTTP 500/000). El sistema logró autoestabilizarse tras aproximadamente 33 segundos (tiempo de vida de las cachés y latidos de Eureka), tras lo cual el balanceador redirigió todo el tráfico a los nodos sanos, reanudando la operación normal sin interrupciones.

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ for i in {1..60}; do
  echo "$(date +%T)  $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL
")"
  sleep 1
done
21:06:56 200
21:06:58 200
21:06:59 200
21:07:01 000
21:07:05 200
21:07:06 200
21:07:08 500
21:07:11 200
21:07:12 500
21:07:16 200
21:07:17 200
21:07:18 200
21:07:20 500
21:07:23 200
21:07:24 200
21:07:26 500
21:07:29 200
21:07:30 500
21:07:34 200
21:07:35 200
21:07:36 200
21:07:38 200
21:07:39 200
21:07:40 200
21:07:42 200
21:07:43 200
21:07:44 200
21:07:46 200
21:07:47 200
21:07:48 200
21:07:49 200
21:07:51 200
21:07:52 200
21:07:53 200
21:07:55 200
21:07:56 200
21:07:57 200
21:07:59 200
21:08:00 200
21:08:01 200
21:08:02 200
21:08:04 200
21:08:05 200
21:08:06 200
21:08:08 200
21:08:09 200
21:08:10 200
21:08:12 200
21:08:13 200
21:08:14 200
21:08:16 200
21:08:17 200
21:08:18 200
21:08:19 200
```

21:45

Ejecución de Flujo Transaccional Completo

Se ejecutó un script de integración automatizado para evaluar la comunicación inter-servicios. Tras simular operaciones financieras complejas, los balances reflejaron exactitud contable: las cuentas finalizaron con saldos de 800/700 tras la transferencia local, y 750/700 tras la simulación de cobro mediante el servicio SPEI.

```
ange]@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ ./demo-v06.sh

== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ==
-> listo en 2 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA          3 instancia(s)
GATEWAY          1 instancia(s)
MOVIMIENTO      1 instancia(s)
NOTIFICACION    1 instancia(s)
SPEI            1 instancia(s)
TRANSFERENCIA   1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe": "002180009160263301", "titular": "Ana", "saldoInicial": 1000}'
{"id":2,"clabe":"002180009160263301","titular":"Ana","saldo":1000}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

== 2. Abrir la cuenta de Beto con 500 (misma puerta) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe": "002180009160263302", "titular": "Beto", "saldoInicial": 500}'
{"id":3,"clabe":"002180009160263302","titular":"Beto","saldo":500}
[HTTP 201]

== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160263301","claveDestino":"002180009160263302","monto":200}'
{"type":"about:blank","title":"Service Unavailable","status":503,"detail":"No se pudo contactar a otro servicio: Could not retrieve response status code: java.net.SocketTimeoutException: Read timed out","instance":"/transferencias"}
[HTTP 503]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-2 GET /cuentas/002180009160263301 atendida por e06a8ee2e291
cuenta-1 GET /cuentas/002180009160263302 atendida por c9e57d875c7d
cuenta-3 POST /cuentas/002180009160263301/cargos atendida por 17ed12c84028
cuenta-2 POST /cuentas/002180009160263302/abonos atendida por e06a8ee2e291
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

== 4. Consultar esa transferencia ==
curl http://localhost:8080/api/transferencias/
{"type":"about:blank","title":"Not Found","status":404,"detail":"No static resource transferencias.", "instance":"/transferencias/"}
[HTTP 404]

== 5. Saldos: el dinero no se crea ni se pierde ==
curl http://localhost:8080/api/cuentas/002180009160263301
{"id":2,"clabe":"002180009160263301","titular":"Ana","saldo":800.00}
[HTTP 200]
curl http://localhost:8080/api/cuentas/002180009160263302
{"id":3,"clabe":"002180009160263302","titular":"Beto","saldo":700.00}
[HTTP 200]
Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700
```

```
== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ==
$ curl http://localhost:8080/api/movimientos?clabe=002180009160263301
[{"id":3,"clabeCuenta":"002180009160263301","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-10T03:24:15.840756Z","referencia":"a 002180009160263301","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T03:24:02.567512Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009160263302
[]
[HTTP 200]

== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ==
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91602633' -d '{"claveOrigen":"002180009160263301","bancoDestino":"BANCO 0000000000009","monto":50}'
{"id":1,"idempotencyKey":"spei-demo-91602633","claveOrigen":"002180009160263301","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91602633' -d '{"claveOrigen":"002180009160263301","bancoDestino":"BANCO 0000000000009","monto":50}'
{"id":1,"idempotencyKey":"spei-demo-91602633","claveOrigen":"002180009160263301","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ==
consulta 1 X-Instancia: c9e57d875c7d (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: 17ed12c84028 (mexi-banco-v06-cuenta-3)
consulta 3 X-Instancia: e06a8ee2e291 (mexi-banco-v06-cuenta-2)
consulta 4 X-Instancia: c9e57d875c7d (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: 17ed12c84028 (mexi-banco-v06-cuenta-3)
consulta 6 X-Instancia: e06a8ee2e291 (mexi-banco-v06-cuenta-2)
consulta 7 X-Instancia: c9e57d875c7d (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: 17ed12c84028 (mexi-banco-v06-cuenta-3)
consulta 9 X-Instancia: e06a8ee2e291 (mexi-banco-v06-cuenta-2)
-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

== 9. Los endpoints internos no tienen puerta ==
$ curl -X POST http://localhost:8080/api/consultas/002180009160263301/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T03:24:41.180+00:00","path":"/api/consultas/002180009160263301/abonos","status":404,"error":"Not Found","requestId":"92d7a29a-574"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

== 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160263301","claveDestino":"002180009160263399","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160263399","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160263301","claveDestino":"002180009160263302","monto":99999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180009160263301","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

== 11. Los saldos no se movieron con los errores ==
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido
```

Comprobación del Balanceo Interno

Analizando los registros (logs) del clúster, se evidenció que los procesos de cargo y abono pertenecientes a una misma transacción financiera fueron procesados por réplicas distintas. Esto valida que la directiva @LoadBalanced implementada habilitó con éxito el descubrimiento y balanceo dinámico de cliente a cliente de forma interna, sin depender de direcciones físicas estáticas.

```
ange]@LAPTOP-STAD6L7D-MINGW64 /d/SD-2027-1/mexi-banco (trabajo-s07)
$ docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-3 | 2026-10-10T03:24:13.271Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160263301/cargos atendida por 17ed12c84028
cuenta-2 | 2026-10-10T03:24:13.480Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160263302/abonos atendida por e06a8ee2e291
cuenta-3 | 2026-10-10T03:24:41.699Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160263301/cargos atendida por 17ed12c84028
cuenta-1 | 2026-10-10T03:24:31.339Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160263301/cargos atendida por c9e57d875c7d
```

Conclusión

La separación de un monolito en microservicios hace que la comunicación sea susceptible a fallos de red, haciendo obligatorio el uso de patrones de resiliencia y descubrimiento. El registro en Eureka abstraer la topología física de la red, permitiendo escalar contenedores horizontalmente sin afectar a los servicios dependientes. El API Gateway unifica el acceso del cliente y actúa como una barrera de seguridad, bloqueando peticiones no autorizadas y enrutando el tráfico hacia los nodos correspondientes. El balanceo de carga Round Robin implementado desde el lado del cliente (con @LoadBalanced) garantiza alta disponibilidad al aislar los contenedores caídos y repartir el procesamiento equitativamente